Welcome to the
Web Application Development with Python L-4
Training Program

Md. Shakil - Assessor & Trainer - National Skill Development Authority (NSDA)

Lecture

09

Python Dictionary

Md. Shakil
Assessor & Trainer
National Skill Development Authority

Python Dictionary

A **dictionary** in Python is a collection of data stored in **key–value pairs**. Dictionaries are **unordered**, **mutable (changeable)**, and **do not allow duplicate keys**. They are widely used to represent real-world data.

dictionary.py

```python
# Creating a dictionary
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
print(person)
```

Why Use Dictionaries?

Dictionaries are used to:

- Store data with meaningful keys
- Access values quickly using keys
- Represent structured data (e.g., student, product, user)
- Map relationships between data

Creating a Dictionary

Dictionaries are created using **curly braces { }** with key–value pairs separated by a colon(:)

```
country_capitals = {
    'Germany' : 'Berlin',     <------ element 1
    'Canada' : 'Ottawa',      <------ element 2
    'England' : 'London'      <------ element 3
}
```

key value

Access Dictionary Elements by Key

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
print(person["name"])   #Output: John
print(person["city"])   #Output: New York
```

Adding Items

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# Adding a new key-value pair
person['email'] = 'alice@example.com'

print(person)
```

Modifying Items

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# Modifying an existing key-value pair
person['age'] = 26

print(person)
```

Find Dictionary Length

dictionary.py

```python
country_capitals = {"England": "London", "Italy": "Rome"}

# get dictionary's length
print(len(country_capitals))   # Output: 2

numbers = {10: "ten", 20: "twenty", 30: "thirty"}

# get dictionary's length
print(len(numbers))            # Output: 3

countries = {}

# get dictionary's length
print(len(countries))          # Output: 0
```

Removing Dictionary Items

Remove Dictionary Items

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# Modifying an existing key-value pair
del person['age']

print(person)
```

pop() Method in Dictionary

Removes and returns the value associated with the specified key. If the key is not found, it raises a KeyError unless a default value is provided.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
removed_value = person.pop('age')
print(removed_value)
print(person)
```

clear() Method in Dictionary

Removes all items from the dictionary.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
print("Before clear: ", person)
person.clear()

print("After clear: ", person)
```

Method in Dictionary

get() Method in Dictionary

Returns the value for a given key. If the key is not found, it returns None.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
data = person.get('age')
print(data)
```

items() Method in Dictionary

Returns a view object that displays a list of dictionary's key-value tuple pairs.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

print(person.items())
```

keys() Method in Dictionary

Returns a view object that displays a list of all the keys in the dictionary.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
dict_keys = person.keys()
print(dict_keys)
```

values() Method in Dictionary

Returns the values in the dictionary.

dictionary.py

```python
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
dict_values = person.values()
print(dict_values)
```

copy() Method in Dictionary

Returns a copy of the specified dictionary.

```
student = {
    "name": "John",
    "age": 20,
    "course": "Python"
}

new_dict = student.copy()
```

Iterate Through a Dictionary

dictionary.py

```python
country_capitals = {
  "United States": "Washington D.C.",
  "Italy": "Rome"
}

# print dictionary keys one by one
for country in country_capitals:
    print("country:",country)


# print dictionary values one by one
for country in country_capitals:
    capital = country_capitals[country]
    print("capital:",capital)
```

Nested Dictionary

dictionary.py

```python
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}

print(myfamily)

print(myfamily["child2"]["name"])
```

Thanks All